

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento:
	Ciencias Exactas
	Carrera: Ing. Electrónica y Automatización Asignatura: Fund. Programación NRC: 28561
	Apellidos y nombres del estudiante:
	Nayerli Montenegro

Ejercicio 56: Funciones void, variables globales y bandera

Objetivo: Comprender funciones void, variables globales y uso de una variable bandera para decidir el mensaje de salida.

```
#include <stdio.h>
int n1,n2,n3;
int band; //variable que sirve como bandera para el artificio
void ingresar()
{
    printf("Ingrese 1 numero: ");
    scanf("%d",&n1);
    printf("Ingrese 2 numero: ");
    scanf("%d",&n2);
}
void imprimir()
{
    if (band==3)
        printf("No existe division para 0\n");
    else
    {
        if (band==1)
            printf("La multiplicacion ");
        else
            printf("La division ");
        printf("entre %d y %d es %d\n",n1,n2,n3);
    }
    /*if (band==1)
        printf("La multiplicacion entre %d y %d es %d\n",n1,n2,n3);
    else
        if (band==2)
            printf("La division entre %d y %d es %d\n",n1,n2,n3);
    else
```

```

        printf("No existe division para 0\n");*/
    }
    void multiplicar()
    {
        n3=n1*n2;
        band=1;
        imprimir();
    }
    void dividir()
    {
        if (n2!=0)
        {
            n3=n1/n2;
            band=2;
        }
        else
            band=3;
        imprimir();
    }
    int main()
    {
        ingresar();
        multiplicar();
        //imprimir();
        dividir();
        //imprimir();
    }

```

Pregunta de análisis: ¿Por qué la función imprimir () no necesita recibir parámetros en este ejercicio?

Porque las variables son declaradas al inicio hace que cualquier función las lea y se las pueda modificarlas.

Ejercicio 57: Menú, mayor y valor absoluto

Objetivo: Aplicar funciones dentro de un menú para calcular el mayor de dos números y el valor absoluto.

```

#include <stdio.h>

int mayor(int num1, int num2) {
    if (num1 > num2) return num1;

    else return num2;
}

```

```

int absoluto(int num) {
    if (num < 0) return -num;
    return num;
}

int main() {
    int opcion;
    int n1, n2;
    int band = 1;
    do {
        printf("\n--- MENU DE OPCIONES ---\n");
        printf("1. Determinar el mayor de dos numeros\n");
        printf("2. Calcular el valor absoluto de dos numeros\n");
        printf("3. Salir\n");
        printf("Seleccione una opcion: ");
        scanf("%d", &opcion);
        if (opcion == 1 || opcion == 2) {
            printf("Ingrese el primer numero: ");
            scanf("%d", &n1);
            printf("Ingrese el segundo numero: ");
            scanf("%d", &n2);
        }
        switch (opcion) {
            case 1:
                printf("El mayor de los dos numeros es: %d\n", mayor(n1, n2));
                break;
            case 2:
                printf("El valor absoluto de %d es: %d\n", n1, absoluto(n1));

```

```

        printf("El valor absoluto de %d es: %d\n", n2, absoluto(n2));

        break;

case 3:

    printf("Saliendo del programa... \n");

    band = 0;

    break;

default:

    printf("Opcion no valida. Intente de nuevo.\n");

    break;

}

} while (band == 1);

return 0;

}

```

Pregunta de análisis: ¿Qué ventaja tiene usar un menú cuando el programa ofrece más de una operación?

Porque de esa manera ordénalos nuestras opciones y podemos dejar de ejecutar cuando lo necesitamos.

Ejercicio 58: Funciones con return y raíz cuadrada

Objetivo: Diferenciar funciones que retornan valores enteros y reales, usando suma y raíz

```

#include <stdio.h>

#include <math.h>

int sumar(int a, int b);

double ingresar();

double raiz();

void imprimir(double resultado);

int main() {

    int resultadoSuma = sumar(10, 9);

    printf("--- OPERACION 1: SUMA (Enteros) ---\n");

    printf("El resultado de la suma directa es: %d\n\n", resultadoSuma);

```

```

printf("--- OPERACION 2: RAIZ CUADRADA (Reales) ---\n");

double resultadoRaiz = raiz();

if (resultadoRaiz != -1) {
    imprimir(resultadoRaiz);
}

return 0;
}

int sumar(int a, int b) {
    return a + b;
}

double ingresar() {
    double num;

    printf("Ingresar un numero positivo para calcular la raiz cuadrada: ");

    scanf("%lf", &num);

    return num;
}

double raiz() {
    double numero = ingresar();

    if (numero < 0) {
        printf("Error: No se puede calcular la raiz cuadrada de un numero negativo.\n");
        return -1;
    }

    return sqrt(numero);
}

void imprimir(double resultado) {
    printf("El resultado de la raiz cuadrada es: %.2f\n", resultado);
}

```

Pregunta de análisis: ¿Por qué sqrt() necesita incluir la librería math.h?

Para que la librería de math.h se pueda usar de esa manera se evita algún error, le dice al programa donde esta la función.

Ejercicio 59: Parámetros y retorno de valores

Objetivo: Usar parámetros para enviar valores a una función y utilizar una función que retorna el menor de dos números.

```
#include <stdio.h>

#include <math.h>

void ingresar();

int mayor(int x, int y);

float menor(float x, float y);

int main() {

    printf("--- 1. PROBANDO INGRESO DE DATOS ---\n");

    ingresar();

    printf("--- 3. PROBANDO FUNCION MENOR() Y SQRT() ---\n");

    float n1 = 16.0, n2 = 25.0;

    printf("El menor entre %.2f y %.2f es: %.2f\n", n1, n2, menor(n1, n2));

    printf("La raiz cuadrada del numero menor es: %.2f\n", sqrt(menor(n1, n2)));

    return 0;

}

void ingresar() {

    float a, b;

    printf("Ingrese el primer numero: ");

    scanf("%f", &a);

    printf("Ingrese el segundo numero: ");

    scanf("%f", &b);

    printf("Valores registrados: %.2f y %.2f\n", a, b);

}

int mayor(int x, int y) {

    if (x > y) {

        return x;
```

```

    } else {
        return y;
    }
}

float menor(float x, float y) {
    if (x < y) {
        return x;
    } else {
        return y;
    }
}

```

Pregunta de análisis: ¿Cuál es la diferencia entre imprimir un resultado dentro de una función y retornarlo al main()?

Ejercicio 60: División con validación

Objetivo: Crear una función de división que reciba dos números reales y valide división por

```

#include <stdio.h>

int dividir_seguro(float a, float b, float *resultado);

int main() {
    float num1, num2, res;

    int exito;

    printf("--- EJERCICIO 60: DIVISION CON VALIDACION ---\n");

    printf("Ingrese el dividendo (numerador): ");

    scanf("%f", &num1);

    printf("Ingrese el divisor (denominador): ");

    scanf("%f", &num2);

    exito = dividir_seguro(num1, num2, &res);

    if (exito == 1) {
        printf("Division exitosa. El resultado es: %.2f\n", res);
    } else {

```

```
        printf("Error: No se puede dividir entre cero (0).\n");
    }

    return 0;
}

int dividir_seguro(float a, float b, float *resultado) {
    if (b == 0) {
        return 0;
    }

    *resultado = a / b;

    return 1;
}
```

Pregunta de análisis: ¿Por qué retornar 0 para indicar error puede generar confusión en una división válida?

Porque esto genera confusión cuando se intenta devolver el resultado de la operación y el estado de error en la misma variable.